

Semana 7
ISIS-1611: Inteligencia Artificial

Carla González

14 de julio de 2026

1. Inferencia en LPO: Cierre

Esta clase cierra la unidad de lógica e inferencia (AIMA Cap. 7–9) antes de pasar a planeación: se repasan model checking en lógica proposicional, encadenamiento hacia adelante y hacia atrás sobre cláusulas de Horn, unificación y el occurs check, y resolución por refutación en LPO. Luego se aplica todo el repaso a una guía de ejercicios completa, resuelta paso a paso.

1.1. Lógica Proposicional: Entailment y Model Checking

Definición 1.1 (Implicación lógica). $KB \models \alpha$ (“ KB implica α ”) si y solo si α es verdadera en *todo* modelo en el que KB es verdadera. Equivalentemente (teorema de refutación):

$$KB \models \alpha \quad \text{si y solo si} \quad KB \wedge \neg\alpha \text{ es insatisfacible.}$$

Model checking (TT-Entails?): para un número finito de símbolos proposicionales, se enumeran las 2^n interpretaciones posibles (tabla de verdad completa), se marcan los **modelos** de KB (filas donde KB es verdadera) y se verifica si α es verdadera en todos ellos. Un solo **contramodelo** (modelo de KB donde α es falsa) basta para refutar $KB \models \alpha$. Es importante distinguir dos nociones que se confunden con frecuencia:

- α es **consistente** con KB : existe *al menos un* modelo de KB donde α es verdadera ($KB \wedge \alpha$ es satisfacible).
- $KB \models \alpha$: α es verdadera en **todos** los modelos de KB .

La primera es mucho más débil que la segunda.

1.2. Cláusulas de Horn y Encadenamiento hacia Adelante

Una **cláusula definida** tiene la forma $P_1 \wedge \dots \wedge P_n \Rightarrow Q$ (a lo sumo un literal positivo). El **encadenamiento hacia adelante** (forward chaining) parte de los hechos conocidos y aplica reglas cuyas premisas ya están satisfechas, derivando nuevos hechos hasta alcanzar la meta o agotar la agenda.

Definición 1.2 (PL-FC-ENTAILS?, con agenda y contadores). Se mantiene, para cada regla, un **contador** con el número de premisas aún no confirmadas, y una **agenda** FIFO de símbolos conocidos verdaderos que faltan por procesar. En cada paso se extrae el siguiente símbolo p de la agenda; si p es la meta, se retorna verdadero; si no, se marca p como inferido y, por cada regla donde p aparece en la premisa, se decrementa su contador; si un contador llega a 0, la conclusión de esa regla se agrega a la agenda.

- **Completo** para bases de conocimiento proposicionales de cláusulas de Horn (deriva exactamente los hechos implicados, ni más ni menos).
- **Dirigido por los datos** (*data-driven*): no usa la meta para decidir qué reglas disparar, así que puede derivar hechos verdaderos pero **irrelevantes** para la consulta.

1.3. Encadenamiento hacia Atrás

El **encadenamiento hacia atrás** (backward chaining) parte de la meta y busca reglas cuya conclusión coincida con ella; cada premisa de esas reglas se convierte, recursivamente, en una nueva **submeta**. Un hecho de la base resuelve una submeta directamente (hoja del árbol de prueba).

- **Dirigido por la meta** (*goal-driven*): solo explora lo relevante para probar la consulta.

- Cuando una regla no logra probar una submeta (no hay hechos ni otra regla aplicable, o todas las alternativas fallan), ocurre **backtracking**: se descarta el intento y se prueba la siguiente regla cuya conclusión coincida con esa submeta.
- Completo para cláusulas de Horn; es la base operacional de Prolog. Sin control de ciclos puede entrar en bucles infinitos si una submeta depende (directa o indirectamente) de sí misma.

1.4. Sustitución, Unificación y Occurs Check

Definición 1.3 (Unificador más general, UMG). $\text{UNIFY}(p, q) = \theta$ tal que $\text{SUBST}(\theta, p) = \text{SUBST}(\theta, q)$, y θ es el **más general** posible: cualquier otro unificador θ' se obtiene componiendo θ con una sustitución adicional. La unificación procede posición a posición: variable con término (se liga la variable), constante con constante idéntica (coincide sin generar ligadura), constantes distintas o predicados/aridades distintas (falla).

Definición 1.4 (Occurs check). Antes de ligar una variable x a un término t , el **occurs check** verifica que x no aparezca dentro de t . Si x ocurriera en t , la ligadura x/t generaría un término **infinito** (p. ej. $x = f(x) = f(f(x)) = \dots$), que no es un objeto válido en LPO. Muchas implementaciones de Prolog omiten el occurs check por eficiencia, arriesgando unificaciones incorrectas que un motor completo rechazaría.

1.5. Forma Clausal y Resolución por Refutación en LPO

Para aplicar resolución en LPO, cada sentencia se convierte a **forma normal conjuntiva (FNC)** mediante los pasos estándar: eliminar $\Rightarrow$ y $\Leftrightarrow$; mover $\neg$ hacia adentro (De Morgan, $\neg\forall x \Phi \equiv \exists x \neg\Phi$, $\neg\exists x \Phi \equiv \forall x \neg\Phi$); **estandarizar variables** (renombrar para que cada cuantificador use una variable distinta); **Skolemizar** (reemplazar cada variable existencial por una constante o función de Skolem de las variables universales que la contienen, según corresponda) y eliminar los cuantificadores universales restantes (quedan implícitos); distribuir $\vee$ sobre $\wedge$.

- **Regla de resolución (LPO):**

$$\frac{\ell_1 \vee \dots \vee \ell_k \quad \neg m_1 \vee \dots \vee \neg m_j}{\text{SUBST}(\theta, \ell_1 \vee \dots \vee m_j)} \quad \theta = \text{UNIFY}(\ell_i, m_i)$$

- **Refutación:** para probar $KB \models \alpha$, se agrega $\neg\alpha$ (en FNC) al conjunto de cláusulas de KB y se aplica resolución hasta derivar la **cláusula vacía** $\square$ (contradicción) —lo que confirma que $KB \wedge \neg\alpha$ es insatisfacible— o hasta no poder generar cláusulas nuevas (en cuyo caso $KB \not\models \alpha$).
- La resolución en LPO es **sana y completa por refutación** (teorema de completitud de Gödel), pero solo **semi-decidible**: si $KB \models \alpha$, se garantiza encontrar $\square$ en un número finito de pasos; si $KB \not\models \alpha$, el procedimiento puede no terminar nunca.

2. Ejercicios: Guía de Lógica e Inferencia (con solución paso a paso)

Esta sección resuelve, de principio a fin, la *Guía de Ejercicios: Lógica e Inferencia*.

2.1. Bloque 1: Preguntas Conceptuales (Verdadero / Falso)

- a. “Si una sentencia α es satisfacible, entonces $KB \models \alpha$ para cualquier base de conocimiento KB consistente con α .”

Falso. Satisfacibilidad de α solo dice que α es verdadera en *algún* modelo; que KB sea *consistente* con α solo dice que $KB \wedge \alpha$ tiene algún modelo. Ninguna de las dos garantiza que α sea verdadera en **todos** los modelos de KB , que es lo que exige $\models$.

Contraejemplo: $KB = \{P\}$, $\alpha = Q$. α es satisfacible, y KB es consistente con α (el modelo $P=T, Q=T$ satisface ambos), pero $KB \not\models Q$ (el modelo $P=T, Q=F$ también es modelo de KB y no de Q).

- b. “El encadenamiento hacia adelante (forward chaining) es completo para bases de conocimiento proposicionales formadas por cláusulas de Horn.”

Verdadero. Para cláusulas de Horn (cada regla con a lo sumo un literal positivo), el algoritmo con agenda y contadores deriva exactamente todos los símbolos atómicos implicados por KB : es sano (solo deriva lo que se sigue lógicamente) y completo (deriva todo lo que se sigue).

- c. “ $KB \models \alpha$ si y solo si la sentencia $(KB \wedge \neg\alpha)$ es insatisfacible.”

Verdadero. Es el **teorema de refutación**, base de todo método de prueba por contradicción (model checking negativo y resolución): si α fuera falsa en algún modelo de KB , ese modelo satisface simultáneamente KB y $\neg\alpha$; si ningún modelo satisface ambos a la vez, entonces α es verdadera en todo modelo de KB .

- d. “Durante la unificación es válido sustituir un término por una constante de Skolem, de la misma forma en que se sustituye una variable.”

Falso. Las constantes (y funciones) de Skolem se introducen *antes* de la unificación, durante la conversión a FNC, para eliminar cuantificadores existenciales; representan un objeto **fijo pero arbitrario** del dominio, no un comodín instanciable. La unificación solo liga **variables** (símbolos que aún pueden tomar cualquier valor) a términos; una constante de Skolem se comporta como cualquier otra constante: puede aparecer *dentro* de un término al que se liga una variable, pero ella misma no puede recibir una ligadura ni ser tratada como si fuera libre de unificarse con otro término distinto.

2.2. Ejercicio 1: El apagón en Paloquemao

Símbolos: P (falló la planta), S (sobrecarga en neveras), T (transformador dañado).

$$R1 : P \vee S \vee T \quad R2 : \neg(P \wedge S) \quad R3 : T \Rightarrow P \quad KB = R1 \wedge R2 \wedge R3$$

P	S	T	$R1$	$R2$	$R3$	KB (i modelo?)
V	V	V	V	F	V	F
V	V	F	V	F	V	F
V	F	V	V	V	V	V (modelo M_1)
V	F	F	V	V	V	V (modelo M_2)
F	V	V	V	V	F	F
F	V	F	V	V	V	V (modelo M_3)
F	F	V	V	V	F	F
F	F	F	F	V	V	F

Cuadro 1: Tabla de verdad completa. KB tiene exactamente tres modelos: $M_1 = (V, F, V)$, $M_2 = (V, F, F)$, $M_3 = (F, V, F)$.

a) Tabla de verdad

b) $KB \models (P \vee S)$? En M_1 : $V \vee F = V$. En M_2 : $V \vee F = V$. En M_3 : $F \vee V = V$. Verdadera en los tres únicos modelos de $KB \Rightarrow KB \models (P \vee S)$.

c) $KB \models S$? En M_1 y M_2 , $S = F$. $KB \not\models S$; contramodelo: $M_1 = (P=V, S=F, T=V)$ (o M_2), donde KB es verdadera pero S es falsa.

d) $KB \models \neg(S \wedge T)$? M_1 : $S \wedge T = F \wedge V = F \Rightarrow \neg(\cdot) = V$. M_2 : $F \wedge F = F \Rightarrow V$. M_3 : $V \wedge F = F \Rightarrow V$. Verdadera en los tres modelos $\Rightarrow KB \models \neg(S \wedge T)$.

e) $KB \models (P \vee T)$? M_1 : $V \vee V = V$. M_2 : $V \vee F = V$. M_3 : $F \vee F = \mathbf{F}$. Como M_3 es un modelo de KB donde $P \vee T$ es falsa, $KB \not\models (P \vee T)$: el administrador **no tiene razón**. M_3 muestra que $P \vee T$ es **consistente** con KB (verdadera en M_1, M_2) pero no está **implicada** por KB (falla en M_3): consistencia solo exige un modelo testigo, mientras que implicación exige que se cumpla en *todos* los modelos de KB , sin excepción.

2.3. Ejercicio 2: Café de especialidad en el Eje Cafetero

$$\begin{array}{lll}
 R1 : L \wedge H \Rightarrow P & R2 : A \wedge C \Rightarrow PE & R3 : P \wedge PE \Rightarrow CE \\
 R4 : CE \Rightarrow LE & R5 : L \wedge A \Rightarrow SO & \text{Hechos: } L, A, C, H
 \end{array}$$

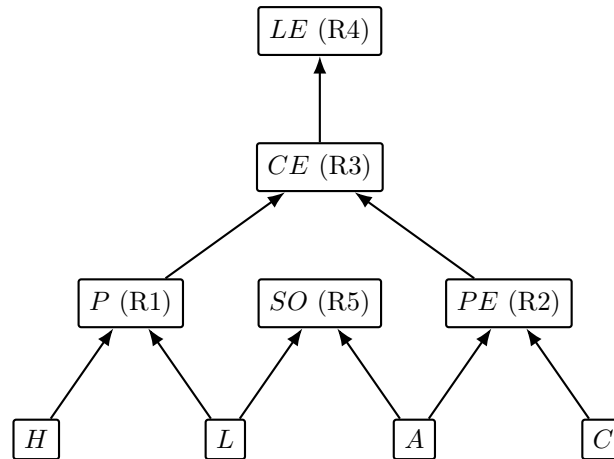


Figura 1: Grafo AND-OR: los hechos L, A, C, H están en la base; cada símbolo derivado agrupa (AND, implícito por la regla que lo produce) las premisas indicadas por los dos arcos que entran a él. Nótese que SO no tiene ningún arco de salida: ninguna regla lo usa como premisa.

a) Grafo AND-OR

Paso	Símbolo procesado	Regla que dispara (cuenta a 0)	Agenda al final del paso
1	L	— (decrementa R1: $2 \rightarrow 1$; R5: $2 \rightarrow 1$)	$[A, C, H]$
2	A	R5 dispara $\rightarrow$ agrega SO (decrementa R2: $2 \rightarrow 1$)	$[C, H, SO]$
3	C	R2 dispara $\rightarrow$ agrega PE	$[H, SO, PE]$
4	H	R1 dispara $\rightarrow$ agrega P	$[SO, PE, P]$
5	SO	— (SO no es premisa de ninguna regla)	$[PE, P]$
6	PE	— (decrementa R3: $2 \rightarrow 1$)	$[P]$
7	P	R3 dispara $\rightarrow$ agrega CE	$[CE]$
8	CE	R4 dispara $\rightarrow$ agrega LE	$[LE]$
9	LE	es la consulta $\rightarrow$ se retorna verdadero	$[]$

Cuadro 2: Traza completa de PL-FC-ENTAILS? para la consulta LE .

b) Traza de forward chaining (agenda FIFO, orden inicial L, A, C, H)

c) **¿Se deriva LE ?** **Sí.** El hecho derivado SO (paso 2, vía R5: $L \wedge A \Rightarrow SO$) **no contribuye en nada** a probar LE : SO no aparece en la premisa de ninguna regla, así que la cadena que lleva a LE ($P, PE \rightarrow CE \rightarrow LE$) nunca lo utiliza.

d) **¿Por qué se deriva un hecho irrelevante?** Forward chaining es **dirigido por los datos**: en cada paso dispara *toda* regla cuyas premisas ya se satisfacen, sin usar información sobre cuál es la consulta. Como L y A (ya conocidos) satisfacen la premisa de R5 independientemente de que SO sirva o no para responder LE , la regla se dispara igual. Esto refleja que el algoritmo es **sano y completo mirando hacia adelante**: deriva absolutamente todo lo que KB implica, sin ningún mecanismo de poda por relevancia a una meta particular.

2.4. Ejercicio 3: La comparsa del Carnaval de Barranquilla

$R1 : I \wedge E \Rightarrow H$ $R2 : H \wedge V \Rightarrow D$ $R3 : G \Rightarrow I$ $R4 : B \Rightarrow I$ Hechos: B, E, V

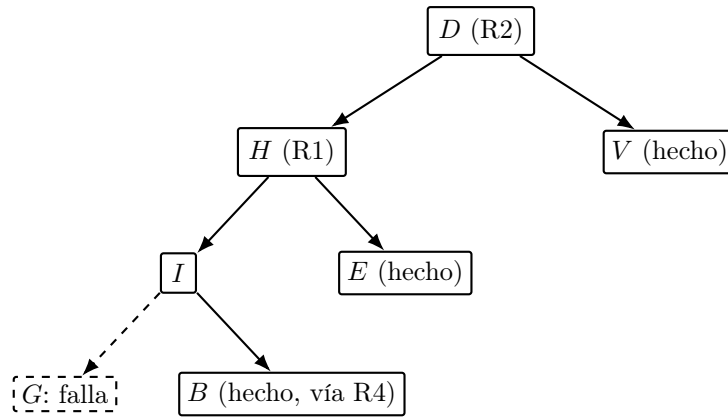


Figura 2: Árbol de prueba de backward chaining para D . El intento de probar I vía $R3$ ($G \Rightarrow I$) falla (rama punteada: G no es hecho ni cabeza de ninguna regla); se hace backtracking y $R4$ ($B \Rightarrow I$) rescata la prueba, ya que B es un hecho.

a) Árbol de prueba para la meta D

b) Submeta con backtracking El backtracking ocurre en la submeta I : como las reglas se intentan en el orden en que aparecen, primero se prueba **R3** ($G \Rightarrow I$), lo que exige probar G ; pero G no es un hecho ni es la conclusión de ninguna otra regla, así que esa rama **falla** sin alternativas. El algoritmo retrocede a la submeta I y prueba la siguiente regla cuya cabeza es I : **R4** ($B \Rightarrow I$), que exige probar B ; como B es un hecho, **R4 rescata la prueba** y I queda demostrado.

c) Comparación con el Ejercicio 2 Forward chaining en el Ejercicio 2 derivó **5 hechos nuevos**: SO, PE, P, CE, LE (a partir de los 4 hechos iniciales). Backward chaining aquí exploró **7 submetas**: D, H, V, I, G, B, E (una de ellas, G , es una rama fallida). Los números son comparables en este ejemplo pequeño, pero la diferencia cualitativa es la que importa:

- **Dirigido por los datos** (forward chaining): parte de los hechos y aplica todas las reglas disparables, sin mirar la meta; puede derivar información verdadera pero irrelevante (como SO). Es preferible cuando **no hay una única meta fija** y se esperan muchas consultas sobre la misma base (p. ej. un sistema de monitoreo que deriva conclusiones a medida que llegan datos nuevos).
- **Dirigido por la meta** (backward chaining): parte de la consulta y solo explora lo que es relevante para probarla. Es preferible cuando la base de conocimiento es **grande** pero solo una fracción pequeña es relevante para una consulta puntual (p. ej. sistemas expertos de diagnóstico, o Prolog respondiendo una pregunta específica), aunque puede explorar y descartar ramas fallidas (como G) antes de encontrar la regla correcta.

2.5. Ejercicio 4: Estación de anillamiento en el páramo

a) Traducción a LPO

a.1. Todas las aves anilladas en Chingaza migran a Sumapaz:

$$\forall x (Ave(x) \wedge Anillada(x, Chingaza)) \Rightarrow Migra(x, Sumapaz)$$

a.2. Existe un ave anillada en Chingaza que cuida su propio nido:

$$\exists x Ave(x) \wedge Anillada(x, Chingaza) \wedge Cuida(x, Nido(x))$$

a.3. Ningún ave migra a la estación donde fue anillada:

$$\forall x \forall e (Ave(x) \wedge Anillada(x, e)) \Rightarrow \neg Migra(x, e)$$

a.4. Toda ave cuida al menos un nido:

$$\forall x Ave(x) \Rightarrow \exists n Cuida(x, n)$$

b) Unificación

b.1. *Anillada(Colibri7, x)* y *Anillada(y, Chingaza)*.

Posición 1: *Colibri7* vs. *y* $\Rightarrow y/Colibri7$. Posición 2: *x* vs. *Chingaza* $\Rightarrow x/Chingaza$.

UMG: $\theta = \{y/Colibri7, x/Chingaza\}$.

b.2. *Migra(x, Nido(x))* y *Migra(Paramuna3, y)*.

Posición 1: *x* vs. *Paramuna3* $\Rightarrow x/Paramuna3$ (occurs check: *x* no ocurre en *Paramuna3*, correcto). Aplicando θ_1 , posición 2: *Nido(Paramuna3)* vs. *y* $\Rightarrow y/Nido(Paramuna3)$ (occurs check: *y* no ocurre en *Nido(Paramuna3)*).

UMG: $\theta = \{x/Paramuna3, y/Nido(Paramuna3)\}$.

b.3. *Vecina(Nido(x), x)* y *Vecina(y, Chingaza)*.

Posición 1: *Nido(x)* vs. *y* $\Rightarrow y/Nido(x)$ (occurs check: *y* $\neq x$, no hay ocurrencia). Posición 2: *x* vs. *Chingaza* $\Rightarrow x/Chingaza$. Componiendo (se sustituye *x* dentro de la ligadura de *y*): *y/Nido(Chingaza)*.

UMG: $\theta = \{x/Chingaza, y/Nido(Chingaza)\}$. Verificación: *Vecina(Nido(Chingaza), Chingaza)* en ambos lados.

b.4. *Anillada(Colibri7, Chingaza)* y *Anillada(Colibri7, Sumapaz)*.

Posición 1: *Colibri7* vs. *Colibri7* (coincide). Posición 2: *Chingaza* vs. *Sumapaz*: dos **constantes distintas**.

No se pueden unificar: la unificación falla porque no existe sustitución que iguale dos constantes diferentes.

b.5. *Cuida(x, Nido(x))* y *Cuida(Nido(y), y)*.

Posición 1: *x* vs. *Nido(y)* $\Rightarrow$ occurs check sobre *x*: *x* no aparece en *Nido(y)* (solo aparece *y*), así que *a priori* pasa: $\theta_1 = \{x/Nido(y)\}$. Aplicando θ_1 a la posición 2: *Nido(x)* se convierte en *Nido(Nido(y))*, que debe unificarse con *y*. Posición 2: *Nido(Nido(y))* vs. *y* $\Rightarrow$ se intentaría *y/Nido(Nido(y))*, pero *y* **ocurre dentro de** *Nido(Nido(y))*.

No se pueden unificar: el occurs check falla en la segunda posición (ligar *y* a un término que contiene a *y* generaría el término infinito $y = Nido(Nido(y)) = Nido(Nido(Nido(y))) = \dots$).

c) Occurs check El **occurs check** es el paso del algoritmo de unificación que, antes de ligar una variable *v* a un término *t*, verifica que *v* **no aparezca dentro de** *t*. De los pares anteriores, solo **b.5** lo requiere (y falla por su causa): al intentar ligar *y* a *Nido(Nido(y))* se crearía un término circular/infinito, que no es un objeto válido en LPO (los términos deben ser finitos). Un motor de inferencia que omita el occurs check (como muchas implementaciones de Prolog, por razones de eficiencia) aceptaría esta unificación y construiría internamente una estructura de datos **cíclica**; esto puede producir inferencias incorrectas (dos términos que en realidad no denotan el mismo objeto se tratan como unificables) o hacer que el motor entre en un **bucle infinito** al intentar recorrer, imprimir o comparar ese término infinito.

2.6. Ejercicio 5: El caso de la pieza extraviada

1. $\forall x (TieneLlave(x) \wedge TurnoNocturno(x)) \Rightarrow Accede(x, Boveda)$
2. $\forall x \forall y (Accede(x, Boveda) \wedge Registro(x, y)) \Rightarrow Movio(x, y)$
3. $TieneLlave(Bruno)$ 4. $TurnoNocturno(Bruno)$ 5. $Registro(Bruno, Poporo)$
6. $TieneLlave(Amparo)$ 7. $TurnoNocturno(Celia)$ $\alpha = \exists x Movio(x, Poporo)$

a) Forma clausal (variables estandarizadas)

$C_1 : \neg TieneLlave(x_1) \vee \neg TurnoNocturno(x_1) \vee Accede(x_1, Boveda)$

$C_2 : \neg Accede(x_2, Boveda) \vee \neg Registro(x_2, y_2) \vee Movio(x_2, y_2)$

$C_3 : TieneLlave(Bruno)$ $C_4 : TurnoNocturno(Bruno)$ $C_5 : Registro(Bruno, Poporo)$

$C_6 : TieneLlave(Amparo)$ $C_7 : TurnoNocturno(Celia)$

Negación de la meta: $\neg \alpha = \neg \exists x Movio(x, Poporo) \equiv \forall x \neg Movio(x, Poporo)$, que en forma clausal (variable universal implícita, estandarizada como x_3) es

$C_8 : \neg Movio(x_3, Poporo).$

Paso	Cláusulas resueltas	MGU	Resolvente
1	C_3 con C_1	$\{x_1/Bruno\}$	$C_9 : \neg TurnoNocturno(Bruno) \vee Accede(Bruno, Boveda)$
2	C_4 con C_9	$\{\}$ (ya coincide en $Bruno$)	$C_{10} : Accede(Bruno, Boveda)$
3	C_{10} con C_2	$\{x_2/Bruno\}$	$C_{11} : \neg Registro(Bruno, y_2) \vee Movio(Bruno, y_2)$
4	C_5 con C_{11}	$\{y_2/Poporo\}$	$C_{12} : Movio(Bruno, Poporo)$
5	C_{12} con C_8	$\{x_3/Bruno\}$	$\square$ (cláusula vacía)

Cuadro 3: Refutación: se deriva $\square$ a partir de $KB \wedge \neg \alpha$.

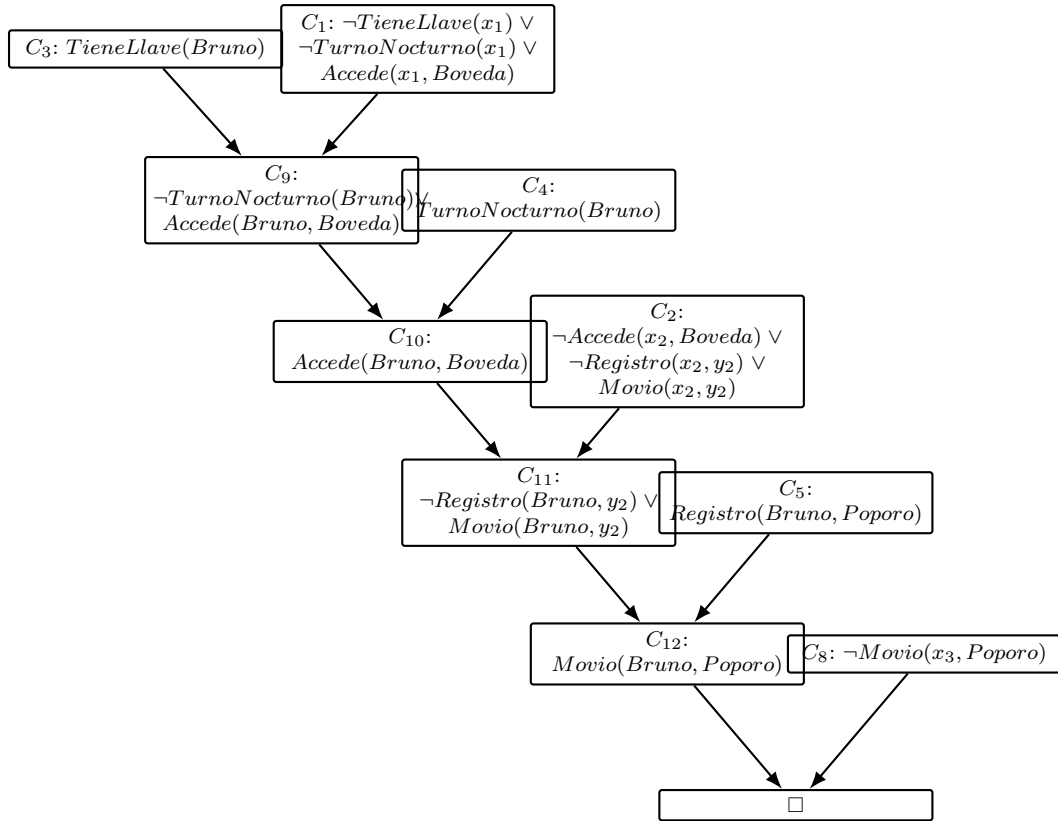


Figura 3: Árbol de refutación completo: cada resolvente combina los dos nodos que apuntan hacia él, hasta llegar a $\square$.

b) Refutación por resolución

c) **Extracción de la respuesta** La variable existencial de la meta original (x en $\exists x \text{ Movio}(x, \text{Poporo})$) se estandarizó como x_3 en C_8 . Acumulando las sustituciones a lo largo de la refutación, x_3 termina ligada a *Bruno* (paso 5). **Respuesta: Bruno movió el poporo.**

d) Por qué Amparo y Celia no cierran una refutación alternativa

- **Amparo** ($C_6: \text{TieneLlave}(\text{Amparo})$): resolviendo con C_1 (x_1/Amparo) se obtiene

$$\neg \text{TurnoNocturno}(\text{Amparo}) \vee \text{Accede}(\text{Amparo}, \text{Boveda}).$$

Para continuar haría falta resolver el literal $\neg \text{TurnoNocturno}(\text{Amparo})$ contra una cláusula unitaria $\text{TurnoNocturno}(\text{Amparo})$, pero esa cláusula **no existe** en la KB (el enunciado aclara que Amparo no estuvo en el turno nocturno): la cadena se detiene ahí.

- **Celia** ($C_7: \text{TurnoNocturno}(\text{Celia})$): resolviendo con C_1 (x_1/Celia) se obtiene

$$\neg \text{TieneLlave}(\text{Celia}) \vee \text{Accede}(\text{Celia}, \text{Boveda}).$$

Aquí falta una cláusula unitaria $\text{TieneLlave}(\text{Celia})$ para resolver $\neg \text{TieneLlave}(\text{Celia})$; tampoco existe (el enunciado aclara que Celia no tiene llave). Igual que con Amparo, no hay literal complementario que unifique.

- Incluso si, hipotéticamente, se lograra derivar $\text{Accede}(\text{Amparo}, \text{Boveda})$ o $\text{Accede}(\text{Celia}, \text{Boveda})$, el segundo paso exigiría resolver contra C_2 y luego necesitaría una cláusula $\text{Registro}(\text{Amparo}, y)$ o $\text{Registro}(\text{Celia}, y)$ para producir Movio ; la única cláusula Registro de la KB es $\text{Registro}(\text{Bruno}, \text{Poporo})$ así que tampoco hay unificador posible ahí.

e) **¿Por qué probar $KB \wedge \neg\alpha$ insatisfacible en vez de derivar α directamente?** Probar $KB \models \alpha$ “directamente” exigiría verificar que α es verdadera en *todo* modelo de KB —en LPO, con dominios potencialmente infinitos, no hay forma de enumerar todos los modelos. La refutación invierte el problema: por el **teorema de refutación** ($KB \models \alpha \Leftrightarrow KB \wedge \neg\alpha$ insatisfacible), basta con encontrar *una* contradicción sintáctica finita (la cláusula vacía $\square$) para concluir que **ningún** modelo puede satisfacer KB y $\neg\alpha$ a la vez, es decir, que todo modelo de KB satisface α . El principio que conecta ambas cosas es precisamente ese teorema de refutación, sustentado por el **teorema de completitud** de la resolución: convierte una pregunta semántica sobre *todos los modelos* en una búsqueda sintáctica, finita y mecánicamente verificable, de una contradicción.

3. Planeación: Definición y Algoritmos Generales

3.1. ¿Qué es la Planeación?

Un agente planificador construye una **secuencia de acciones** para lograr un objetivo en un entorno modelado.

Definición 3.1 (Problema de Planeación STRIPS/PDDL). ■ **Estado:** conjunción de fluentes (literales positivos/negativos).

- **Acción:** definida por precondiciones y efectos.
- **Meta:** conjunción de fluentes que deben ser verdaderos.

3.2. Representación PDDL

Listing 1: Ejemplo de acción en PDDL

```

1 (:action Fly
2   :parameters (?p - Plane ?from ?to - Airport)
3   :precondition (and (At ?p ?from) (Plane ?p)
4                     (Airport ?from) (Airport ?to))
5   :effect (and (At ?p ?to) (not (At ?p ?from))))

```

3.3. Algoritmos de Búsqueda para Planeación

- **Búsqueda hacia adelante** (*progression*): empieza en el estado inicial y aplica acciones.
- **Búsqueda hacia atrás** (*regression*): empieza desde la meta y aplica acciones inversas.
- **SATPLAN**: codifica el problema de planeación como SAT y usa un solucionador SAT.

4. Planeación: Heurísticas y Planeación Jerárquica

4.1. Heurísticas para Planeación

4.1.1. Problema Relajado (ignore preconditions / ignore delete lists)

- **Ignore preconditions:** todas las acciones son aplicables. Heurística: número mínimo de acciones para satisfacer la meta.
- **Ignore delete lists:** no se eliminan fluentes. Problema de planeación relajado resoluble en tiempo polinomial.

4.1.2. Planning Graph y Heurística FF

Un **grafo de planeación** alterna capas de literales y acciones:

- Capa de literales S_i : literales alcanzables en i pasos.
- Capa de acciones A_i : acciones aplicables en el paso i .

La heurística FF usa el plan del grafo relajado como estimador.

4.2. Planeación Jerárquica (HTN)

Hierarchical Task Network (HTN): descompone tareas abstractas en subtareas más concretas.

- **Tareas primitivas**: acciones directamente ejecutables.
- **Tareas abstractas**: se descomponen mediante métodos.
- Ventaja: permite incorporar conocimiento de dominio para guiar la búsqueda.

5. Planeación en Ambientes No Deterministas

5.1. Ambientes No Deterministas y Condicionales

Cuando las acciones tienen efectos inciertos, se necesitan **planes condicionales** que incluyan ramificaciones según el resultado de las acciones.

- **Plan de búsqueda AND-OR**: los nodos AND representan contingencias del entorno; los nodos OR representan elecciones del agente.
- Un plan condicional es una solución si cubre todas las ramas.

5.2. Planeación Parcialmente Observable

Con observabilidad parcial el agente mantiene un **estado de creencia** (belief state) y ejecuta acciones de recopilación de información.

5.3. Planeación Continua y de Ejecución

- **Monitoreo de ejecución**: detectar discrepancias entre el plan y la realidad.
- **Replaneación**: adaptar el plan ante fallos o nueva información.
- **Agentes reactivos-deliberativos**: combinan planeación a largo plazo con respuesta reactiva.